arXiv:2208.07220v2 [cs.CV] 4 Oct 2022

PatchDropout: Economizing Vision Transformers Using Patch Dropout

Yue Liu [1,2] *, Christos Matsoukas [1,2,5], Fredrik Strand [3,4], Hossein Azizpour [1], Kevin Smith [1,2]

[1] KTH Royal Institute of Technology, Stockholm, Sweden [2] Science for Life Laboratory, Stockholm, Sweden
[3] Karolinska Institutet, Stockholm, Sweden [4] Karolinska University Hospital, Stockholm, Sweden
[5] AstraZeneca, Gothenburg, Sweden

Abstract

Vision transformers have demonstrated the potential to outperform CNNs in a variety of vision tasks. But the computational and memory requirements of these models prohibit their use in many applications, especially those that depend on high-resolution images, such as medical image classification. Efforts to train ViTs more efficiently are overly complicated, necessitating architectural changes or intricate training schemes. In this work, we show that standard ViT models can be efficiently trained at high resolution by randomly dropping input image patches. This simple approach, PatchDropout, reduces FLOPs and memory by at least 50% in standard natural image datasets such as IMAGENET*, and those savings only increase with image size. On* CSAW*, a high-resolution medical dataset, we observe a* $5\times$ *savings in computation and memory using PatchDropout, along with a boost in performance. For practitioners with a fixed computational or memory budget, PatchDropout makes it possible to choose image resolution, hyperparameters, or model size to get the most performance out of their model.*

1. Introduction

Vision Transformers (ViTs) [5] have been recently introduced as a viable alternative to CNNs [5, 12, 13, 23]. However, promises of better performance have not yet been realized in many settings due to computational bottlenecks. For instance, ViTs require large datasets to train on [5], though this issue has been partially solved using pre-training on large datasets [5, 1]. Memory and compute requirements add to this, since the self-attention mechanism introduces an element with quadratic complexity *w.r.t.* the number of tokens. These bottlenecks can result in long training times, and for large images such as those encountered in medical image analysis, the computational and memory demands render off-the-shelf ViTs unsuitable.

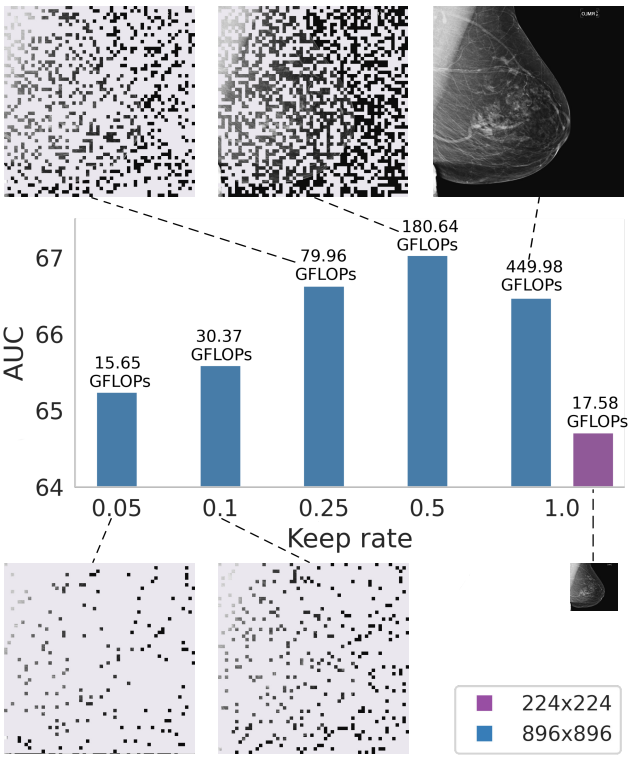

Figure 1: *PatchDropout can be used to efficiently train off-the-shelf ViTs on high-resolution images.* A tiny proportion of the input patches is often enough for accurate prediction, and if larger resolution images are used performance can increase, despite the lost information. Here, training with different ratios of input patches affects the model's performance on CSAW, a real-world medical dataset with high resolution images. PatchDropout results in increased predictive performance under the same computational budget. Using a $16\times$ larger image but keeping 5% of the patches saves computation and memory, and improves performance. Further improvements can be achieved by increasing the keep rate, at the expense of computation. A similar trend is observed across standard image classification datasets.

*Corresponding author: Yue Liu <yue3@kth.se>

These computational issues are acute in other domains as well, *e.g.* microscopy and remote sensing, especially when native resolution is not only a desired property but a requirement for accurate predictions. Accordingly, several works focus on making vision transformers more efficient using a plethora of different approaches, which usually involve some kind of post-processing or architectural modifications [19, 31, 32, 26, 33]. These methods prioritize efficiency during inference, *e.g.* for embedding in mobile devices, and have been shown to reduce run-time by 30 to 50% without compromising performance. However, the bottleneck in network training can not be overlooked. Few works have addressed this topic, and those that do require architecture modifications or complex training schemes which limits their use [29, 12, 30, 20, 22]. Efficient ViT training remains an important problem, especially for applications requiring large images, as all but the largest institutions are limited by computational resources to train ViTs.

In this work, we ask a fundamental question. *Are all input patches necessary during training, or can we randomly disregard a large proportion of them?* An affirmative response entails a simple, yet efficient approach that reduces compute and memory footprint. Our method, PatchDropout, randomly drops input tokens and results in up to $5\times$ reduction in memory and compute during training when using high-resolution images, without compromising the model accuracy (Figure 1). This can be achieved with off-the-shelf vision transformers and a minimal implementation, owing to the nature of ViTs. Furthermore, we show that given a fixed memory and computational budget, PatchDropout makes it possible to choose image resolution, hyperparameters, or model size to get the most performance out of the model. We conduct experiments on CSAW, a real-world medical dataset with high resolution images, and further validate our proposed method using three mainstream datasets: IMAGENET, CIFAR100 and PLACES365. Through these experiments we show that:

- We can randomly discard image patches during training without compromising performance and improve efficiency from $2\times$ up to $5.6\times$, depending on image size (see Figures 1 and 5).
- Given the same computational budget, up-scaling the images and/or utilising a larger ViT variant while discarding a fraction of input tokens can improve the model's accuracy (see Table 3 and Table 4).
- PatchDropout can act as a regularization technique during training, resulting in increased model robustness (see Figure 6).

These findings along with additional ablation studies suggest that PatchDropout can economize ViTs, allowing their utilization on high-resolution images, with potential gains in accuracy and robustness. Code to reproduce our work is available at https://github.com/yueliukth/PatchDropout.

2. Related Work

Several studies have examined how to obtain a lighter vision transformer model using an existing well-trained one to improve inference efficiency using *e.g.* pruning or a teacher for distillation. DynamicViT [19] adds a prediction module for estimating the importance score of each patch progressively. The training is assisted by knowledge distillation and the patches whose contribution are minimal to the final prediction are pruned during inference. Another pruning method, PatchSlimming [26] identifies less important patches from the last layer and removes them from previous ones. DVT [31] dynamically determines the number of patches by training a cascade of transformers using an increasing number of patches and then interrupts inference once the prediction is confident.

Another line of research focuses on making the training more efficient. Several studies attempt optimization of network architectures through artificially designed modules [29, 12, 30], among which PatchMerger [20] and TokenLearner [22] are designed specifically for reducing the number of tokens. EViT [10] learns to gradually preserve the attentive tokens and fuse the inattentive ones during training which results in a 0.3% decrease in accuracy on IMAGENET with a 50% increase in speed of inference. Compared to EViT, the proposed method of this study is complementary but with a much simpler mechanism that does not require substantial modifications.

A few recent works explore the possibility of learning expressive representations by selecting a subset of patches. MAE [6], which is designed for more efficient self-supervised pre-training, proposes dropping a high proportion of patches and subsequently inferring the missing patches through an autoencoder setup. Our work takes some inspiration from MAE, however, PatchDropout can be applied to target tasks directly using standard ViTs (unlike MAE). In [8], the authors augment standard ViTs with additional patches that selectively attend to a subset of patches to improve transferability of ViTs. Finally, [17] shows that ViTs are robust to random occlusions. However, it should be noted that occlusion does not result in efficiency improvement.

3. Methods

Transformer models were originally developed for language-related tasks [28], but their self-attention mechanism has been proven useful for vision tasks as well [5, 27, 12]. An important difference between the two tasks

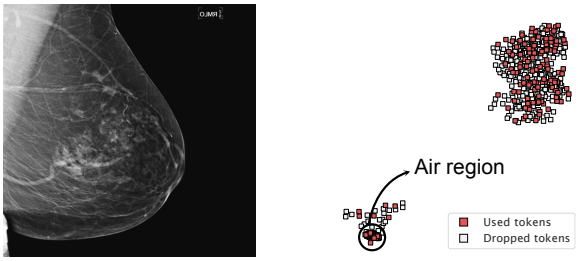

Figure 2: *Redundancy in mammographic images.* **(Left)** An example image from CSAW. **(Right)** 2D projection of the extracted patches of the left image using UMAP [16]. The red squares represent the randomly kept patches after PatchDropout with a keep rate of 0.25. The patches are clustered into 2 distinct groups: the air and the breast regions. Surprisingly, by simply sampling uniformly, the information needed for accurate classification is retained.

is that visual data, often, contains considerable redundancy or correlation in appearance [6] (see Figure 2). This observation leads to the following question: *Can we randomly omit input image patches during training? If yes, what are the benefits of doing so?* Here, we aim to answer these questions, showing that vision transformers can indeed be trained using a fraction of the input data and perform well, while at the same time saving a significant amount of memory and compute. Additionally, our simple training scheme may offer some desirable regularization effects.

3.1. PatchDropout

Our core idea relies on the fact that the spatial redundancy encountered in image data can be leveraged to economize vision transformers. If we randomly deny a fraction of the information to the model during training, we expect a diminished impact on the model's predictive performance. PatchDropout implements this by randomly dropping a percentage of image tokens at the input level (see Figure 3). More specifically, before the patch embeddings are sent to transformer blocks, a subset of tokens is randomly sampled without replacement. Positional embeddings are added prior to the random sampling so that the corresponding position information is retained. The `[CLS]` token is retained if it exists. The sampled token sequence is sent to transformer blocks in the standard manner. The proposed method is straightforward and trivial to implement, which makes it viable to be incorporated in most ViT models without substantial modifications.

3.2. Complexity Analysis

Vision transformers operate on a series of tokens, where each token corresponds to a non-overlapping image patch and is represented by a linear projection of the patch summed with a positional embedding. In practice, an image of size $H \times W$ is tiled into $N = HW/P^2$ patches,

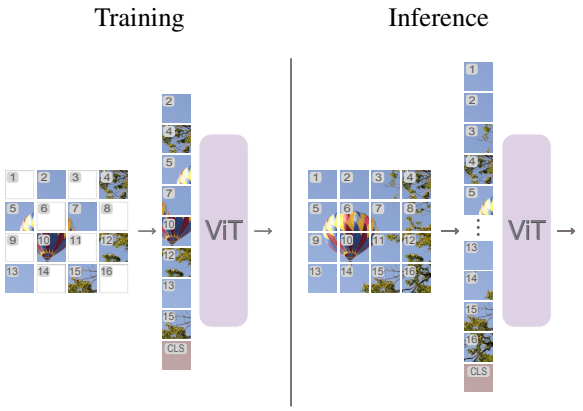

Figure 3: *PatchDropout during training and inference.* **(Left)** PatchDropout is easy to implement. Patchify the image and add positional embeddings to each patch. Uniformly sample a subset of them and use them to train the model. **(Right)** At test time, all patches are retained.

where P is the patch size and it is typically defined by the user (often 8 or 16). The resulting token sequence is fed into a series of consecutive transformer blocks that update the d-dimensional embedding of tokens and consist of a Multi-head Self-Attention (MSA) and Multi-Layer Perceptron modules (MLP). The MSA itself includes a series of MLP layers that model the interactions between the tokens through attention. A final MLP layer is responsible for projecting the output to have the same dimensions as its input, ready to be processed by the next transformer block. Given this information, we can discuss the theoretical and empirical computational complexity of vision transformers.

Theoretical complexity Given L transformer blocks with N tokens and d-dimensional embeddings, the computational cost of the self-attention within the MSA module is $\mathcal{O}(LN^2d)$, while the other MLP layers introduce a complexity of $\mathcal{O}(LNd^2)$. In total, the computational complexity of a series of L transformer blocks is:

$$2LN^2d + 4LNd^2. \tag{1}$$

The compute is always linear to the depth L. When $N \gg d$ the complexity reduces to the first term, when $N \ll d$ it reduces to the second term.

For high resolution images with small patch sizes, which is the focus of this work, the first term prevails. This leads to a quadratic complexity with respect to the sequence length N. Accordingly, removing a non-trivial portion of the input tokens can result in significant savings in compute.

Empirical complexity In practice, the observed computation cost may not exactly reflect the theoretical prediction. A few factors can make the computational saving of PatchDropout less advantageous than the complexity analysis might suggest. For instance, the patchifier, *i.e.* the layer

responsible for tokenizing and projecting the input image into a series of embedded tokens adds computational overhead. The same is true for the classification head. Nonetheless, as image size increases (and thus the input sequence length N increases), the gap between the theoretical and empirical relative computation should lessen. This necessitates an empirical analysis of the savings in computation to confirm the theoretical predictions.

In Figure 4 we illustrate the relative drop in computations, both according to Eq. 1 and empirically according to the number of FLOPs. We compare for different sequence lengths N when using two keep rates of 0.5 and 0.25. As discussed above, the computational saving of PatchDropout increases with increasing number of tokens N. It can be seen that the drop in computation is similar to the keep rate for small N, but gradually converges to a quadratic savings with increasing N. While the theoretical and empirical trends are similar, the empirical saving converges slower due to the additional computations discussed above. Note that N varies with image size $H \times W$ and patch size P. The default image size across multiple vision benchmark datasets is 224×224. At this scale, the relative computation stays close to the value of its keep rate – the images are not large enough to benefit much from the quadratic savings. However, many real-world tasks demand high-resolution images, like the ones typically encountered in the medical domain. Here, we observe large computational savings as the sequence length increases.

Finally, an important factor to note is that the embedding dimensions d varies between different ViT variants, affecting their computational savings. In general, the relative computation of PatchDropout on smaller ViTs (with smaller d) decreases faster compared to larger ones.

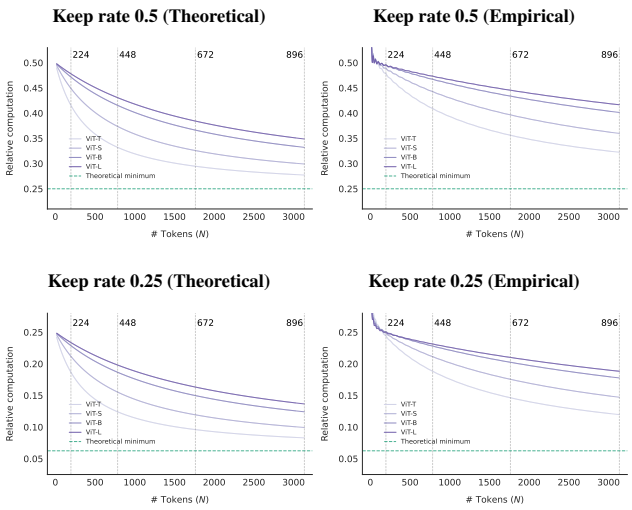

Figure 4: *Computational savings with PatchDropout increase for larger sequence length N.* We illustrate the theoretical (left) and empirical (right) relative savings in computation when using PatchDropout with keep rates 0.5 and 0.25 for different input sequence lengths and ViT models. Different sized images with a fixed patch size 16 result in different numbers of tokens (vertical dashed lines). As discussed in Section 3.2, empirical savings do not always correspond to the theoretical analysis due to various factors. However, the trend remains consistent: as the image size, and thus the number of input tokens N increases, the observed computational savings approach the theoretical minimum $\lim_{N\to\infty}$.

4. Experimental Setup

We evaluate PatchDropout across a number of different ViT variants and datasets. As representative ViT models, we selected DeiTs [27] of different capacity and SWINs [12], which are ViT variants that scale linearly with respect to the input sequence length by design. For datasets, we selected three standard benchmark image classification datasets and a real-world medical dataset of high-resolution images. Below, we describe the experimental settings in detail, and in Section 5 we report our findings.

Data selection In this work we attempt to economize vision transformers such that they can be utilized for tasks where high-resolution images are necessary for accurate predictions. To this end, we select a subset of 190,094 high-resolution images from CSAW, a population based cohort which consists of millions of mammography scans primarily developed for breast cancer tasks [2, 15, 24]. Here, we focus on the breast cancer risk prediction, a sensitive classification task. The data is split at the patient-level and the validation set contains balanced classes, resulting in 152,922 training images, 3,256 validation images, and 33,916 testing images. Furthermore, to validate the applicability of PatchDropout in other domains and on conventional image sizes, we run experiments on 3 standard image classification datasets: ImageNet [3], CIFAR100 [9] and Places365 [34]. Adhering to standard practice, we report our results on the official validation splits of ImageNet and Places365 and we use 1% of the training data for validation. On CIFAR100, 2% of the training images comprise the validation set and the results are reported on the official test set.

Preprocessing Images from CSAW are in DICOM format, and require several pre-processing steps which are detailed below. Using the DICOM metadata, we re-scale the intensity values and correct any images with inverted contrast. Following [11], certain images are excluded according to a set of exclusion criteria. The purpose is to filter out noisy images, images with implants, biopsies and mammograms with aborted exposure. The mammograms with

Input	Keep rate	Memory (GB)	GFLOPs	AUC
224	1	1.46	17.58	64.71%
896	0.05	1.50	15.65	65.27%
896	0.10	1.65	30.37	65.59%
896	0.25	2.51	79.96	66.63%
896	0.50	5.15	180.64	67.03%
896	1	14.86	449.98	66.47%

Table 1: *Performance (AUC), memory and compute savings using PatchDropout on* CSAW. The memory is computed with a batch size of 1 on a single GPU.

cancer signs are separated from those intended for risk estimation. More specifically, cases with examination 60 days in advance of diagnosis are excluded in order to avoid risk conflation. For the other datasets we only resize their images to meet the needs of our experiments, no additional pre-processing was performed. Further details are provided in Supplementary A.

Models and training protocols In this study, we primarily use DEITs [27] which are similar in spirit and computational complexity to the original ViTs [5]. Unless otherwise specified, the model choice is DEIT-B trained on 16×16 patches (denoted as DEIT-B/16), and it is trained on input size 224×224. Additionally, to show that PatchDropout is agnostic to architectural selections, we run ablations using SWINs [12]. SWINs designed to reduce the computational complexity of the original ViTs. They scale linearly with respect to the input size and they inherit some of the CNN's inductive biases by design. Additional implementation details can be found in Supplementary B.

5. Results and Discussion

We begin this section by demonstrating that not all input patches are necessary during training – hence, we can randomly discard a large proportion of them. Then, we show how PatchDropout can be used not only to save memory and compute but also to improve the model's predictive performance. Finally, we analyze the regularization effects of PatchDropout and its role as an augmentation method. Unless otherwise stated, each experiment is repeated 3 times and we report the mean value of the appropriate metric for each dataset. For IMAGENET, CIFAR100 and PLACES365, we report top-1 accuracy and for CSAW the exam-level AUC, where the predictions take the average score of each mammogram in an examination.

Are all input patches necessary during training? To assess the impact of PatchDropout and determine whether all tokens are necessary for training ViTs, we conduct experiments where different percentages of the tokens are presented to the model. As illustrated in Figure 1 and Table 1, 25% of the tokens are enough to train an accurate model on high-resolution CSAW images of 896×896 pixels, while consuming more than 80% less memory and compute. Interestingly, models trained with 25% or 50% of the tokens outperform a model that uses all tokens. This hints at a regularization effect for PatchDropOut that we will discuss later.

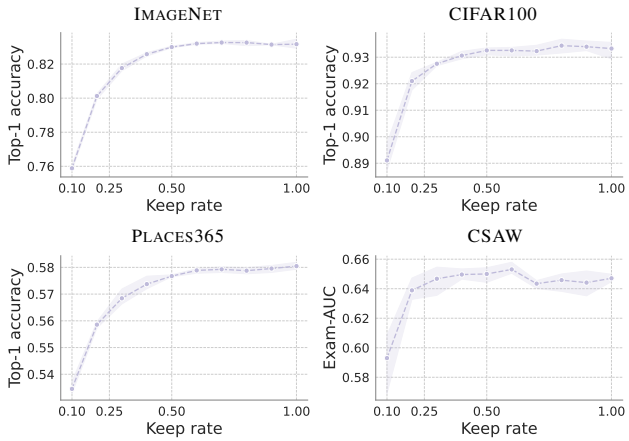

Figure 5: *Not all input patches are necessary to be present.* 50% of the input patches are sufficient to preserve model performance for image size 224×224: it improves efficiency $2\times$ while the performance drop is contained at only 0.17% on IMAGENET, 0.07% on CIFAR100 and 0.38% on PLACES365. On CSAW, keeping around half of the input patches results in 0.25% - 0.60% increase in AUC compared with keeping all tokens.

Keep rate	Memory (GB)	GFLOPs	IMAGENET	CIFAR100	PLACES365	CSAW
1	20.96	17.58	83.17%	93.33%	58.05%	64.71%
0.9	0.89×	0.90×	−0.03%	+0.07%	−0.09%	−0.30%
0.8	0.78×	0.79×	+0.09%	+0.11%	−0.17%	−0.13%
0.7	0.68×	0.69×	+0.09%	−0.10%	−0.12%	−0.37%
0.6	0.57×	0.59×	+0.03%	−0.07%	−0.16%	+0.60%
0.5	0.48×	0.50×	−0.17%	−0.07%	−0.38%	+0.29%
0.4	0.39×	0.40×	−0.59%	−0.27%	−0.68%	+0.25%
0.3	0.30×	0.40×	−1.41%	−0.58%	−1.21%	−0.04%
0.2	0.22×	0.20×	−3.04%	−1.23%	−2.19%	−0.82%
0.1	0.14×	0.10×	−7.28%	−4.22%	−4.60%	−5.41%

Table 2: *Performance, memory and compute savings using PatchDropout on various datasets with* 224×224 *images.*

In Figure 5 and Table 2 we explore how this trend translates to standard image classification benchmark datasets using 224×224 images. We observe that performance varies as a function of the keep rate. In all cases, a keep rate of 50% or larger is sufficient to maintain good performance. When using exactly 50% of the tokens, the performance drop is contained to only 0.17% on IMAGENET, 0.07% on CIFAR100 and 0.38% on PLACES365. The reduction in memory and computation are significant and similar to the keep rate.

Input	Patch	Keep rate	GFLOPs	IMAGENET	CIFAR100	CSAW
64	16	1	1.46	66.78%	87.27%	-
64	8	0.25	1.46	70.57%	89.77%	-
128	16	0.25	1.49	**76.25%**	**91.30%**	-
112	16	1	4.33	77.65%	91.98%	63.07%
112	8	0.25	4.33	79.11%	92.38%	60.08%
224	16	0.25	4.41	**81.02%**	**92.50%**	**64.87%**
224	16	1	17.58	83.17%	**93.33%**	64.71%
224	8	0.25	17.58	**83.43%**	92.71%	64.28%
448	16	0.25	17.93	83.26%	92.20%	**65.59%**
448	16	1	78.57	-	-	66.31%
448	8	0.25	78.57	-	-	66.13%
896	16	0.25	79.96	-	-	**66.63%**

Table 3: *Effect of varying image size and patch size.* The impact in terms of FLOPS and performance of changing the image size and patch size is measured using PatchDropout over multiple datasets.

Can we trade the savings in memory and compute introduced by PatchDropout for more accurate predictions? In the previous analysis we saw that PatchDropout allows for significant memory and computational savings without compromising the model's performance. This saving can enable a more elaborate model selection (*e.g.* finer grid search) or a wider range of training choices (*e.g.* larger batch size) or a more accurate but computationally-heavy architecture. Therefore, the next question we ask is whether we could utilize the saved memory and compute to improve the model's predictive performance while keeping the computational budget similar to the one used for the full token sequence. Our experiments show that this can be easily achieved with two simple design choices:

(1) Increasing the total token sequence by (a) using higher resolution images, or (b) decreasing the patch size.

(2) Employing models with greater capacity.

– Larger images or smaller patch size It has been proven that ViTs perform better on larger images and smaller patch sizes [5, 27, 1]. However, this comes with a large memory and computational overhead due to the increased input sequence length, as described in Section 3. PatchDropout mitigates this cost by reducing the sequence length, allowing for the utilization of larger images and smaller patches. Table 3 illustrates the trade-off between the model's performance and the input sequence length for different settings on various datasets.

Trading the saved compute from PatchDropout for larger images yields a large performance boost for almost all setups (compare the 1st and 3rd row of each group). For example, comparing IMAGENET for images of size 128×128 with keep rate of 0.25 with the 64×64 images with all tokens retained, we find that this simple trade-off results in a nearly 10% absolute increase in accuracy for similar cost. The performance gains lessen, however, for larger computational budgets. This trend is observed across all the datasets.

Model	Keep rate	Memory (GB)	GFLOPS	IMAGENET	CIFAR100	CSAW
DEIT-T	1	5.06	1.26	75.22%	86.94%	63.45%
DEIT-S	0.25	2.46	1.15	**78.09%**	**90.30%**	**63.76%**
DEIT-S	1	10.20	4.61	80.69%	91.08%	64.62%
DEIT-B	0.25	5.46	4.41	**81.02%**	**92.50%**	**64.87%**
DEIT-B	1	20.96	17.58	83.17%	93.33%	64.71%
DEIT-L	0.25	15.34	15.39	**83.81%**	**93.97%**	**65.31%**

Table 4: *Impact of training larger ViT variants with PatchDropout using* 224×224 *images.*

Model	Depth	Keep rate	Memory (GB)	GFLOPS	IMAGENET	CIFAR100	CSAW
DEIT-B	12	1	20.96	17.58	**83.17%**	93.33%	64.71%
DEIT-B	24	0.5	19.73	17.31	83.06%	**93.40%**	**65.42%**
DEIT-B	48	0.25	20.95	17.31	81.46%	92.71%	65.31%

Table 5: *Impact of training deeper models with PatchDropout using* 224×224 *images.*

Trading the savings obtained using PatchDropout for smaller patch sizes also yields significant performance gains. However, the gains are not as consistent as for resolution (compare the 1st and 2nd row of each group). For the natural domain, smaller patch sizes improve model performance, as expected from [5, 27]. On CSAW, smaller patch size seems to negatively affect performance, but the effect diminishes as we move to higher resolutions.

Note that, in some cases, images are up-sampled in our experiments. In general, we notice that higher resolution and smaller patch size is usually beneficial, but not always. We speculate that as we move away from a dataset's native resolution, larger input size and smaller patch sizes might have a negative impact on model performance due to significant information loss. Nevertheless, PatchDropout allows for the exploration of hyperparameter settings unfeasible to reach with the full token sequence.

– Models with larger capacity Increasing the model capacity is another way to attain better predictions. The memory and compute saved from PatchDropout can be spent on training larger ViT variants. In Table 4, we explore this trade-off. Interestingly, the trend is that larger models using PatchDropout are consistently better than the smaller variants of equivalent cost that use all tokens. This trend follows across all data domains, with memory efficiency improving up to $2.1\times$. Natural datasets win bigger gains with PatchDropout using larger models, as compared to increased image size or reducing token size.

An alternative way to increase the model capacity is to stack more transformer blocks to a particular ViT variant. We explore this trade-off for a fixed computational budget by increasing the model's depth and varying the keep rate. We report the results in table 5. When doubling the model's transformer blocks with PatchDropout we observe

#Networks	Keep rate	GFLOPs	IMAGENET	CIFAR100	CSAW
1	1	17.58	83.17%	93.33%	64.71%
2	0.5	17.44	**83.48%**	**93.74%**	**65.26%**
4	0.25	17.66	82.20%	93.45%	64.92%

Table 6: *Using PatchDropout compute savings to train an ensemble.*

performance boosts for CIFAR100 and CSAW. However, we note worse performance on IMAGENET when the model becomes too deep. We attribute this to the fact that ViT architectures are optimized for IMAGENET.

Ensembles of models are yet another alternative to obtain more accurate predictions. We explore how computational savings from PatchDropout can be spent training additional models for use in an ensemble. In Table 6, we show that an ensemble of two networks trained with 50% keep rate consistently outperforms a single model without PatchDropout. However, the gains diminish when lower keep rates (25%) are traded for an ensemble for four networks.

Can PatchDropout be used as a regularisation method? Previously, we noticed that PatchDropout can result in increased performance compared to the same settings but with all tokens, *e.g.* in Figure 5. This has some implications about its regularization effects. Thus we ask: *(i)* Can PatchDropout be used as a regularizer? and *(ii)* Does PatchDropout provide robustness against information removal?

To answer the first question we run experiments treating PatchDropout as an augmentation method. In detail, at each iteration we uniformly sample a keep rate between 0.5 and 1 which we use to randomly select a subset of image patches. We report the results in Table 7 and we conclude that PatchDropout is a useful augmentation method. It provides regularization in the sense that generalization is improved across all the datasets. This is not entirely surprising, as PatchDropout behaves similarly to known CNN regularization methods, like cutout [4].

If PatchDropout has some regularization benefits, a further question is: *can it provide robustness against information loss?* To address this question, we evaluate models that have been trained with all image patches and models that have been trained using PatchDropout with different keep rates. During test time, we randomly remove image content using different keep rates. Our results are presented in Figure 6. We find that, in all cases, models that have been trained with PatchDropout exhibit increased robustness against information removal (the green curve is consistently above the blue curve). For completeness, we also report the curves when using all tokens at test time for models that have been trained with PatchDropout (purple curve in Figure 6). These results further validates the regularization effects of our method.

Keep rate	IMAGENET	CIFAR100	CSAW
1	83.17%	93.33%	64.71%
{0.5,1}	**83.32%**	**93.57%**	**65.04%**

Table 7: *PatchDropout has regularization properties.* Rather than using all tokens, training ViTs with PatchDropout using random keep rates improves generalization.

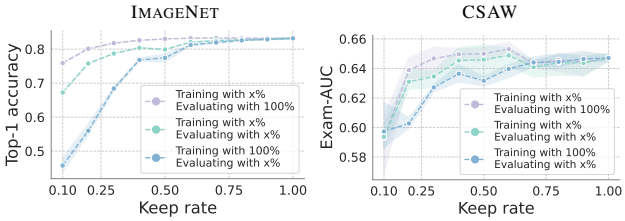

Figure 6: *PatchDropout improves model robustness.* We deny information to the model during inference by randomly dropping input patches and measuring the change in performance. The green curve shows the model's performance when training with a percentage of the input patches and evaluating using the same keep rate. The blue curve represents the model's performance when training using all patches but evaluating on a subset of the input patches. For completeness, the purple curve shows training with PatchDropout and inference with all patches. When 50% or more of the patches are kept, this results in a minimal performance drop on IMAGENET and increased predictive performance on CSAW. The trends show that models trained with PatchDropout are more robust to missing information during inference.

Is PatchDropout constrained by the architectural choice? Throughout our work we used the DEIT model family as they are the most suitable for the purpose of our analysis. This however, raises the question of whether PatchDropout is effective for other architectural choices. To answer this question we run experiments using SWINs [12] which are models purposely designed to reduce the computational complexity of DEITs. SWINs operate using a window shifting approach and re-assignment of positional embeddings at each block. This, necessitates a slightly different implementation of PatchDropout. Instead of randomly sampling image patches, we apply structured sampling where we randomly sample column and row indices for each window to obtain the intersection tokens. This maintains the spatial relationships between tokens and enables smooth window shifting for SWINs. The corresponding relative positional biases are sampled accordingly.

We report our findings in Figure 7 when using 224×224 images for both IMAGENET and CSAW. We discern similar patterns with the ones in Figure 5. CSAW exhibits small performance gains when using PatchDropout with keep rates larger than 50% while IMAGENET displays a 1%

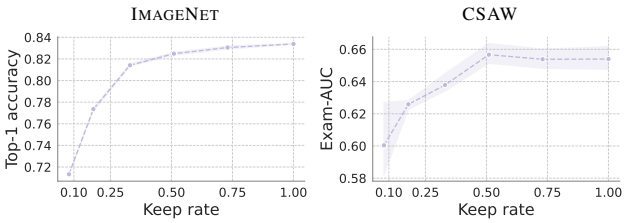

Figure 7: *PatchDropout also works for SWINs.* Despite their linear scaling via the reintroduction of CNN inductive biases, PatchDropout can be applied to SWINs with a keep rate of 0.5 or higher without lowering performance.

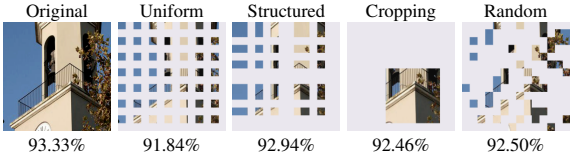

Figure 8: *Impact of different patch sampling strategies.* Accuracy on CIFAR100 is reported for keep rate 0.25 at 224×224 resolution for various patch sampling strategies.

drop at 50% keep rate. The applicability of PatchDropout however is still valid, even for this architecture which has been developed to economize DEITs by design.

Other ablation studies We conclude our analysis with two ablation studies aiming to assess the effectiveness of random sampling and the role of large scale pretraining.

– How important is the sampling strategy in PatchDropout? To assess the efficacy of random sampling, which we use in our method, compared to other sampling methods, we conduct a small ablation study on CIFAR100 where we train models using a fixed keep rate of 0.25 but we change the sampling method. In Figure 8 we illustrate the sampling methods we used and we report the top-1 accuracy. The results show that the effectiveness of the proposed method is not heavily dependent on the choice of sampling strategy. This, along with the results from SWINs where we used structured sampling (see previous paragraph) indicates that PatchDropout is a general approach which can be easily incorporated into other types of ViT models.

– Is PatchDropout sensitive to the initialization strategy? Throughout this work we utilized models pretrained on IMAGENET-21K as vision transformers rely on large-scale pretraining, especially when working with small datasets [5, 27, 13, 14]. But, *is PatchDropout useful when random initialization [7] is used?* To answer this question we train randomly initialized models on CSAW and we report the results in Table 8. Indeed, PatchDropout works with randomly initialized models on CSAW, although with diminishing performance gains, suggesting that the proposed method is agnostic to the initialization strategy.

Keep rate	IMAGENET-21K init.	Random init.
1	64.71%	59.32%
0.50	+0.29%	+0.16%
0.25	+0.16%	−0.90%

Table 8: *Impact of* IMAGENET-21K *initialization for PatchDropout on* CSAW.

6. Conclusion

In this work, we rely on the fact that the spatial redundancy encountered in image data can be leveraged to economize vision transformers and we propose a simple yet efficient method, PatchDropout. By dropping input tokens at random, our method results in significant memory and computation reduction, especially on high-resolution images. In addition, we demonstrate how the saved compute introduced by PatchDropout can be exchanged for better predictive performance under the same memory and computational budget. Finally, we show that PatchDropout can act as a regularization technique during training, resulting in increased model robustness. PatchDropout requires minimal implementation and works with off-the-shelf vision transformers. We believe that PatchDropout should be an essential tool in every practitioner's toolkit to reduce the memory and computational demands in transformer training.

Broader Impact Reducing computational requirements can help to democratize deep learning by making it cheaper to train models. Achieving an equitable outcome for vulnerable and disadvantaged groups, who might lack access to sufficient funding and resources – including but not limited to small academic groups, hospitals, and companies, necessitates a multitude of solutions. Apart from social benefits, one might consider the positive impacts with regards to climate as well. Data centers worldwide account for a substantial portion of energy consumption and GHG emissions which can be mitigated by shrinking computation during model development if applied widely. Despite our efforts, training state-of-the-art networks such as ViTs is still computationally expensive and thus has significant carbon footprints.

Acknowledgements. This work was partially supported by MedTechLabs (MTL), the Swedish Research Council (VR) 2017-04609, Region Stockholm HMT 20200958, and Wallenberg Autonomous Systems Program (WASP). The computations were enabled by the Berzelius resource provided by the Knut and Alice Wallenberg Foundation at the National Supercomputer Centre.

References

[1] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9650–9660, 2021.

[2] Karin Dembrower, Peter Lindholm, and Fredrik Strand. A multi-million mammography image dataset and population-based screening cohort for the training and evaluation of deep neural networks—the cohort of screen-aged women (csaw). *Journal of digital imaging*, pages 1–6, 2019.

[3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.

[4] Terrance DeVries and Graham W Taylor. Improved regularization of convolutional neural networks with cutout. *arXiv preprint arXiv:1708.04552*, 2017.

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[6] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.

[8] Hao Hu, Federico Baldassarre, and Hossein Azizpour. Learnable masked tokens for improved transferability of self-supervised vision transformers. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2022.

[9] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[10] Youwei Liang, Chongjian Ge, Zhan Tong, Yibing Song, Jue Wang, and Pengtao Xie. Not all patches are what you need: Expediting vision transformers via token reorganizations. *arXiv preprint arXiv:2202.07800*, 2022.

[11] Yue Liu, Hossein Azizpour, Fredrik Strand, and Kevin Smith. Decoupling inherent risk and early cancer signs in image-based breast cancer risk models. In *International conference on medical image computing and computer-assisted intervention*, pages 230–240. Springer, 2020.

[12] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10012–10022, 2021.

[13] Christos Matsoukas, Johan Fredin Haslum, Magnus Söderberg, and Kevin Smith. Is it time to replace cnns with transformers for medical images? *arXiv preprint arXiv:2108.09038*, 2021.

[14] Christos Matsoukas, Johan Fredin Haslum, Moein Sorkhei, Magnus Söderberg, and Kevin Smith. What makes transfer learning work for medical images: Feature reuse & other factors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9225–9234, 2022.

[15] Christos Matsoukas, Albert Bou Hernandez, Yue Liu, Karin Dembrower, Gisele Miranda, Emir Konuk, Johan Fredin Haslum, Athanasios Zouzos, Peter Lindholm, Fredrik Strand, et al. Adding seemingly uninformative labels helps in low data regimes. In *International Conference on Machine Learning*, pages 6775–6784. PMLR, 2020.

[16] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[17] Muhammad Muzammal Naseer, Kanchana Ranasinghe, Salman H Khan, Munawar Hayat, Fahad Shahbaz Khan, and Ming-Hsuan Yang. Intriguing properties of vision transformers. *Advances in Neural Information Processing Systems*, 34:23296–23308, 2021.

[18] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.

[19] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: Efficient vision transformers with dynamic token sparsification. *Advances in neural information processing systems*, 34:13937–13949, 2021.

[20] Cedric Renggli, André Susano Pinto, Neil Houlsby, Basil Mustafa, Joan Puigcerver, and Carlos Riquelme. Learning to merge tokens in vision transformers. *arXiv preprint arXiv:2202.12015*, 2022.

[21] Meta Research. fvcore: Flop counter for pytorch models. https://github.com/facebookresearch/fvcore/.

[22] Michael S Ryoo, AJ Piergiovanni, Anurag Arnab, Mostafa Dehghani, and Anelia Angelova. Tokenlearner: What can 8 learned tokens do for images and videos? *arXiv preprint arXiv:2106.11297*, 2021.

[23] Fahad Shamshad, Salman Khan, Syed Waqas Zamir, Muhammad Haris Khan, Munawar Hayat, Fahad Shahbaz Khan, and Huazhu Fu. Transformers in medical imaging: A survey. *arXiv preprint arXiv:2201.09873*, 2022.

[24] Moein Sorkhei, Yue Liu, Hossein Azizpour, Edward Azavedo, Karin Dembrower, Dimitra Ntoula, Athanasios Zouzos, Fredrik Strand, and Kevin Smith. Csaw-m: An ordinal classification dataset for benchmarking mammographic masking of cancer. *arXiv preprint arXiv:2112.01330*, 2021.

[25] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.

[26] Yehui Tang, Kai Han, Yunhe Wang, Chang Xu, Jianyuan Guo, Chao Xu, and Dacheng Tao. Patch slimming for efficient vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12165–12174, 2022.

[27] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357. PMLR, 2021.

[28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[29] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[30] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 568–578, 2021.

[31] Yulin Wang, Rui Huang, Shiji Song, Zeyi Huang, and Gao Huang. Not all images are worth 16x16 words: Dynamic transformers for efficient image recognition. *Advances in Neural Information Processing Systems*, 34:11960–11973, 2021.

[32] Huanrui Yang, Hongxu Yin, Pavlo Molchanov, Hai Li, and Jan Kautz. Nvit: Vision transformer compression and parameter redistribution. *arXiv preprint arXiv:2110.04869*, 2021.

[33] Hongxu Yin, Arash Vahdat, Jose M Alvarez, Arun Mallya, Jan Kautz, and Pavlo Molchanov. A-vit: Adaptive tokens for efficient vision transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10809–10818, 2022.

[34] Bolei Zhou, Agata Lapedriza, Aditya Khosla, Aude Oliva, and Antonio Torralba. Places: A 10 million image database for scene recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2017.

Supplementary Material for PatchDropout: Economizing Vision Transformers Using Patch Dropout

A. Preprocessing

The images from CSAW are resized to 896×896, 448×448, 224×224 and 112×112 for the purposes of this work. The image resolution on IMAGENET varies and has an average resolution of 469×387. For PLACES365 we used images of size 256×256 and for CIFAR100 32×32. We resize these to 448×448, 224×224, 128×128, 112×112 and 64×64 in our experiments using bi-linear interpolation.

B. Models and training protocols

All models are initialized from IMAGENET-21K pre-trained weights and subsequently fine-tuned on the target task. Hyper-parameters are selected through grid search based on the result from the validation set. We use early stopping. The batch size is consistently 128 for all experiments, and each model is trained with an SGD optimizer with momentum set to 0.9. In every experiment, a linear learning rate warmup is utilized for the first 2 epochs. The learning rates are 3×10^{-4} on IMAGENET, 5×10^{-4} on CIFAR100 and PLACES365, and 10^{-4} on CSAW, following the results of our grid search. We use a weight decay of 1×10^{-4} and label smoothing [25] of 0.1. We apply randomly resized cropping, random horizontal flipping, random rotation and color jittering as augmentations. Unless otherwise specified, each experiment was repeated three times and we report the mean. All the experiments are conducted with PyTorch [18]. The number of FLOPs is counted using the fvcore toolkit [21], and the allocated memory is calculated on a single GPU with a batch size of 128, unless otherwise stated.